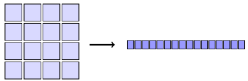
Advanced Artificial Intelligence and Machine Learning

Transformers Outside NLP



Learning goals

- Vision Transformers: how an image becomes a token sequence, and why CNN's inductive biases (locality, translation equivariance) are a data-efficiency trade-off, not a hard requirement
- Making ViT practical: windowed/hierarchical attention (Swin) and training without massive data via distillation (DeiT)
- DETR and MaskFormer: object detection and segmentation as set prediction with a non-autoregressive decoder – a third encoder-decoder pattern alongside T5's and Whisper's autoregressive ones
- Two ways to combine modalities: contrastive dual-encoders (CLIP, incl. its actual training objective) vs. fusion architectures, and where image captioning and text-to-image generation fit

Recap and Roadmap

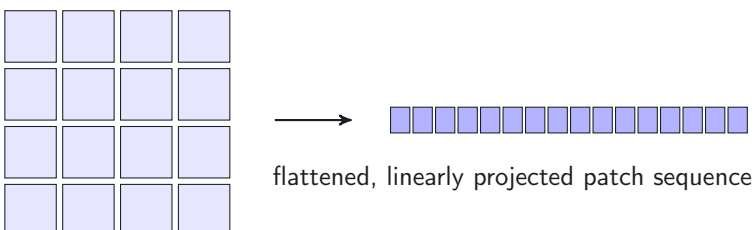
WHERE WE LEFT OFF

- Sessions 7-9 covered the Transformer's three families – encoder-only, decoder-only, encoder-decoder – entirely through their NLP flagships: BERT, GPT, and T5/the original Transformer.
- Session 8 already showed brief evidence this isn't NLP-specific (Vision Transformer, CLIP, Flamingo, GPT-4, Whisper). This session goes deep on *how* those and related non-NLP models actually work.
- Running order: Vision Transformers (encoder-only, applied to images) $\rightarrow$ DETR (a third encoder-decoder pattern) $\rightarrow$ multimodal architectures (combining modalities, two different ways) $\rightarrow$ unified architectures (one model, any modality) $\rightarrow$ video, briefly $\rightarrow$ a final synthesis across all four Transformer sessions.

Vision Transformers, In Depth

FROM PIXELS TO TOKENS

- ViT's central move (Dosovitskiy et al., 2020): split an image into fixed-size, non-overlapping patches (e.g. 16×16 pixels), flatten each patch, and linearly project it – exactly the token-embedding step from session 7, just applied to pixel patches instead of sub-word tokens.



image, split into patches

flattened, linearly projected patch sequence

FROM PIXELS TO TOKENS / 2

- A learnable `[CLS]` token is prepended (exactly BERT's convention), and a learnable position embedding is added to every patch (position is otherwise lost, per the permutation-equivariance property from session 7 – 2D images need it just as much as 1D text does).
- The result feeds an entirely standard Transformer **encoder** – bidirectional self-attention, no architectural changes from session 7 at all. The `[CLS]` token's final representation drives classification, again exactly like BERT.

FROM PIXELS TO TOKENS / 3

Concretely, with real numbers: a standard 224×224-pixel image, split into 16×16 patches:

- Patches per row/column: $224/16 = 14$. Total patches: $14 \times 14 = 196$.
- Plus the `[CLS]` token: a sequence of **197 tokens** enters the encoder – about the length of two or three typical sentences in session 7's terms, not the thousands of pixels the raw image has.
- Each patch is $16 \times 16 \times 3 = 768$ raw numbers (RGB), linearly projected down to whatever embedding size d_{model} the model uses (768 for the original "ViT-Base", not a coincidence – it matches BERT-Base's embedding size exactly).

FROM PIXELS TO TOKENS / 4

- Intuition for *why* patches at all, instead of one token per pixel: at pixel-level granularity a 224×224 image would be a 50,176-token sequence, and self-attention's $\mathcal{O}(n^2)$ cost (session 7) makes that hopeless. Patches are the coarsest grouping that still lets the model see the whole image, chosen purely to keep the sequence short enough to attend over.

CNNS VS. VIT: INDUCTIVE BIAS

- **Inductive bias**: the built-in assumptions a model architecture makes before seeing any data.
- CNNs bake in two strong ones: **locality** (pixels mainly interact with nearby pixels) and **translational equivariance** (a shifted object still gets recognized the same way, because the same kernel slides over the whole image).
- ViT has neither built in – self-attention connects every patch to every other patch equally from the start, with no notion of "nearby" unless the model learns one.

CNNS VS. VIT: INDUCTIVE BIAS / 2

Making this concrete: imagine a photo of a cat, once with the cat centered, once with the exact same cat shifted into the bottom-right corner.

- A CNN's convolutional kernel is a small filter that slides across the *entire* image using the *same* weights everywhere – so whatever pattern it learned to detect (an ear, a whisker) is detected wherever it occurs. It never had to see a bottom-right-corner cat during training to recognize one at test time; the sliding-window mechanism gives it that for free.

CNNS VS. VIT: INDUCTIVE BIAS / 3

- A ViT patch in the bottom-right corner has its *own* position embedding, entirely separate from the center patches' embeddings. If the model only ever saw centered cats during training, there's no built-in reason a bottom-right cat should be recognized the same way – the model would have to have *learned*, from examples, that patch content matters more than patch position for this task.
- That's the whole trade-off in one picture: a CNN gets "it doesn't matter where in the image this pattern appears" for free, by construction. A ViT has to discover that fact from data – and until it has seen enough varied examples to discover it, it will be worse at exactly this kind of generalization than a CNN is by default.

CNNS VS. VIT: INDUCTIVE BIAS / 4

- Below a certain dataset size, CNNs win: their inductive biases are simply correct priors for natural images, and ViT has to spend data learning what a CNN gets for free.
- Above that threshold, ViT catches up and surpasses CNNs – self-attention's lack of built-in assumptions becomes an advantage once there's enough data to learn the *right* assumptions instead of a fixed one, and Transformer blocks scale (in compute and in expressiveness) better than convolutions do.
- Practical consequence: very few people train a ViT from scratch. The standard workflow is transfer learning from a model already pretrained on a massive dataset – freeze most weights, fine-tune only the classification head (or, for a bigger domain gap, fine-tune the whole model at a low learning rate).

CNNS VS. VIT: INDUCTIVE BIAS / 5

Framing after Hugging Face's Community Computer Vision Course, "Vision Transformers for Image Classification" (MIT licensed); Dosovitskiy et al., 2020.

Efficient Vision Transformers

SWIN TRANSFORMER: WINDOWED ATTENTION

- Plain ViT attends every patch to every other patch – the same $\mathcal{O}(n^2)$ cost from session 7, now over however many patches a (potentially high-resolution) image needs.
- **Swin Transformer** (Liu et al., 2021) restricts self-attention to small, non-overlapping local windows (e.g. 7×7 patches): patches only attend within their own window, not globally – linear complexity in image size instead of quadratic. This is session 7's **local attention** pattern, used here as the default design, not an optional efficiency add-on.

SWIN TRANSFORMER: WINDOWED ATTENTION / 2

- To still let information cross window boundaries, alternating layers use a **shifted** window partition – the windows in layer $l+1$ are offset from layer l's, so patches near a boundary in one layer end up in the same window in the next.

SWIN TRANSFORMER: WINDOWED ATTENTION / 3

An intuition for shifting: imagine reading a book one paragraph at a time, understanding each paragraph fully before moving to the next (that's one layer of fixed windows) – but an idea that happens to span a paragraph *break* never gets connected, because the two halves are never read together.

- Swin's fix: in the very next layer, shift where the "paragraph breaks" fall (the window boundaries), so content that was split across two windows before is now inside the same window.
- Stack enough of these shifted layers and information has effectively traveled across the whole image, one window-hop at a time – without ever paying for a single patch to attend to every other patch directly.

SWIN TRANSFORMER: WINDOWED ATTENTION / 4

- This is a cheaper, structured way to reach the same goal Transformer-XL's segment recurrence solved for text (session 7): let information cross a hard boundary the current layer can't see across, without going back to full quadratic attention.

SWIN TRANSFORMER: WINDOWED ATTENTION / 5

- **Hierarchical structure**: between stages, a `PatchMerging` layer merges groups of neighboring patches and doubles the channel depth – exactly the resolution-shrinks-while-channels-grow pattern a CNN's pooling layers produce, which plain ViT has no equivalent of (it keeps one fixed resolution throughout).
- Consequence: Swin produces a multi-scale feature pyramid, so it plugs directly into detection/segmentation architectures designed around CNN backbones – one reason it (not plain ViT) became the default Transformer backbone for those tasks. Swin V2 scales to 3B parameters.

TRAINING VIT WITHOUT MASSIVE DATA: DEIT

- The pain point from two frames ago: ViT needs pretraining on hundreds of millions of images (the original paper used JFT-300M) to beat a CNN. Most labs don't have that.
- **DeiT** (Touvron et al., 2021) trains a competitive ViT on ImageNet-1k alone ($\sim$1.3M images) via **knowledge distillation** from a CNN teacher.
- Mechanism: add a second learnable token – a **distillation token** – alongside the `[CLS]` token. `[CLS]` trains on the true labels as usual; the distillation token trains to match the CNN teacher's prediction. Both attend to the same patch sequence throughout the network.

TRAINING VIT WITHOUT MASSIVE DATA: DEIT / 2

- Same soft/hard-target distillation idea as DistilBERT (session 8) – but the goal here isn't a smaller model, it's transferring a CNN's locality/translation-equivariance bias into a data-hungry architecture that has none of its own.

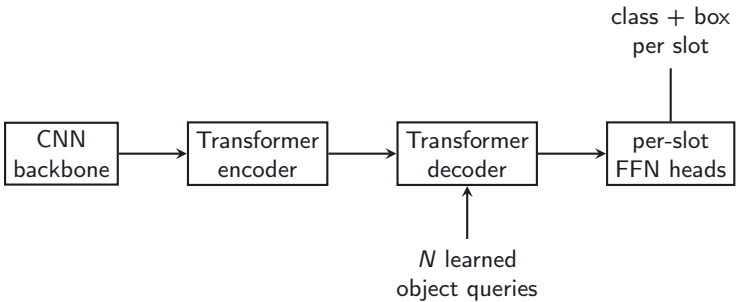
DETR: A Third Encoder-Decoder Pattern

OBJECT DETECTION, THE OLD WAY

- **Task**: given an image, output a set of (class, bounding box) pairs – e.g. `[(car, x, y, w, h), (pedestrian, x', y', w', h')]`.
- Classical detectors (e.g. YOLO): hand-crafted **anchor boxes** as position/size priors, dense per-cell predictions, then heavy post-processing (non-maximum suppression) to remove duplicate/overlapping boxes.
- **DETR** (Carion et al., 2020) removes all of that: a CNN backbone plus a Transformer encoder-decoder predicts the whole set of boxes directly, in parallel, with (almost) no post-processing.

DETR'S DECODER ISN'T AUTOREGRESSIVE

- Every encoder-decoder example so far (T5, Whisper) generates its output *autoregressively*, one token at a time. DETR's decoder does not.



DETR'S DECODER ISN'T AUTOREGRESSIVE / 2

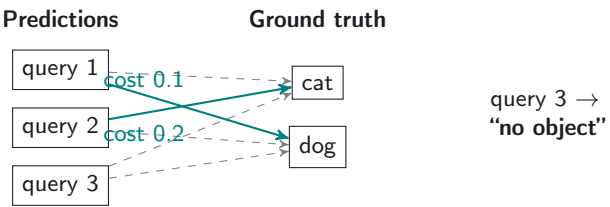
- A fixed number N of learned **object query** vectors (e.g. $N = 100$) are decoded *simultaneously* via cross-attention over the encoder's output – one query per potential object, all at once, no left-to-right dependency between them.
- Most queries predict a "no object" class; the rest predict a real (class, box) pair.

MATCHING PREDICTIONS TO GROUND TRUTH: BIPARTITE MATCHING

- **Question:** If predictions come out unordered and in parallel, which predicted box should be compared against which ground-truth box when computing the loss?
- DETR finds the one-to-one assignment (a **bipartite matching**) between the N predictions and the ground-truth objects that minimizes total matching cost (class mismatch + box mismatch), via the Hungarian algorithm – then computes an ordinary loss on that specific pairing.
- Because the assignment is permutation-invariant by construction, DETR never needs anchor boxes or non-maximum suppression: exactly one prediction is trained to match each ground-truth object.

MATCHING PREDICTIONS TO GROUND TRUTH: BIPARTITE MATCHING / 2

A small worked example. Say $N = 3$ queries predicted boxes for a photo with 2 real objects (a cat, a dog); one query will be assigned "no object".



MATCHING PREDICTIONS TO GROUND TRUTH: BIPARTITE MATCHING / 3

- Every possible pairing (all dashed *and* solid lines) has a cost; the Hungarian algorithm picks the one-to-one assignment (teal, solid) with the *lowest total cost* – here, query 1 $\rightarrow$ dog and query 2 $\rightarrow$ cat, even though query 1 happened to be listed first and "cat" happened to be listed first; the algorithm doesn't care about either ordering, only about the total cost.
- Query 3 matches nothing well enough, so it's assigned the "no object" class – exactly the outcome that lets DETR handle images with anywhere from zero to N objects using one fixed-size output.

DETR IS SURPRISINGLY SHORT CODE

- A ResNet-50 backbone, a single `1x1` conv to reduce channels, PyTorch's built-in `nn.Transformer`, and two linear heads – the "no anchors, no NMS" simplicity claim is visible directly in the code, not just in prose.
- `query_pos`: exactly the $N = 100$ learned object queries from the previous frame, as a plain trainable parameter matrix.

Code: Hugging Face Community Computer Vision Course, "DETR" (MIT licensed), itself adapted from Carion et al., 2020's own minimal implementation.

DETR'S EVOLUTION

- Original DETR: slow to converge, weaker on small objects.
- **Deformable DETR** (Zhu et al., 2020): each query attends only to a small, learned set of sampling points near a reference point (not every encoder position) – cheaper, and multi-scale feature maps (as in YOLOv3) fix the small-object weakness.
- **Conditional DETR** (Meng et al., 2021): splits the cross-attention query into a content part and a spatial part instead of adding them together, isolating "what" from "where" – reported 6.7× faster convergence.

Segmentation Transformers

FROM BOXES TO PIXELS: MASKFORMER

- Classification (ViT) $\rightarrow$ detection (DETR) $\rightarrow$ **segmentation** completes the trio of core vision tasks – and the same set-prediction idea completes the trio too.
- Classical CNN segmentation (e.g. Mask R-CNN): hand-designed, task-specific components – region proposals, per-class mask heads – with genuinely different architectures for semantic, instance, and panoptic segmentation.

FROM BOXES TO PIXELS: MASKFORMER / 2

- **MaskFormer** (Cheng et al., 2021) reframes segmentation as DETR-style set prediction: a pixel-level module extracts per-pixel features, a Transformer decoder computes a *fixed set* of per-segment embeddings from learned **mask queries** (the direct analog of DETR's object queries), and each one is turned into a (class, binary mask) pair.

FROM BOXES TO PIXELS: MASKFORMER

/ 3

- Because each mask query can represent either "all pixels of class X" (semantic) or "this one specific instance" (instance/panoptic), the same architecture handles all three segmentation variants – one unified framework instead of three separate ones.
- Like ViT and DETR, this is data-hungry and hard to train from scratch: the typical recipe freezes a pretrained backbone and decoder, fine-tuning only the Transformer module for the target dataset.

Framing after Hugging Face's Community Computer Vision Course, "Vision Transformers for Image Segmentation" (MIT licensed).

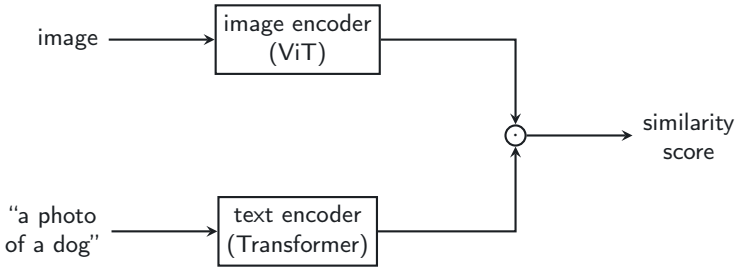
BASIC CV TASKS: ONE RECIPE, THREE INSTANCES

	ViT	**DETR**	**MaskFormer**
Task	classification	detection	segmentation
Output	one label	N (class, box) pairs	N (class, mask) pairs
Query	`[CLS]` token	object queries	mask queries
Decoder	none (encoder only)	parallel, non-autoregressive	parallel, non-autoregressive

- Same underlying move throughout: replace a task's hand-designed, task-specific machinery (anchor boxes, region proposals, per-class mask heads) with a learned set of query vectors and an otherwise ordinary Transformer.

Combining Modalities: Two Architectural Patterns

PATTERN 1: CONTRASTIVE DUAL ENCODERS (CLIP)



- **Two separate encoders**, one per modality, each producing a single embedding vector – no cross-attention between them at all.

PATTERN 1: CONTRASTIVE DUAL ENCODERS (CLIP) / 2

- **Contrastive training** (CLIP, Radford et al., 2021): given a batch of (image, caption) pairs, train both encoders so that each image's embedding has high dot-product similarity with *its own* caption's embedding, and low similarity with every other caption in the batch.
- Result: a shared embedding space where "zero-shot classification" is just nearest-neighbor lookup – embed the image once, compare against embeddings of candidate class names, no task-specific head or fine-tuning needed.
- Fast at inference (embed once, compare with dot products) but cannot model fine-grained interaction *between* the two inputs – it only ever compares two whole-sequence summaries.

CLIP'S TRAINING OBJECTIVE, CONCRETELY

- Take a batch of N (image, caption) pairs; embed all N images and all N captions, and compute the full $N \times N$ matrix of pairwise similarity scores (image embedding $\cdot$ text embedding).
- Train with a **symmetric contrastive loss**: for each image (row), the correct caption should score highest among all N captions; for each caption (column), the correct image should score highest among all N images – an N-way softmax cross-entropy in each direction, averaged.

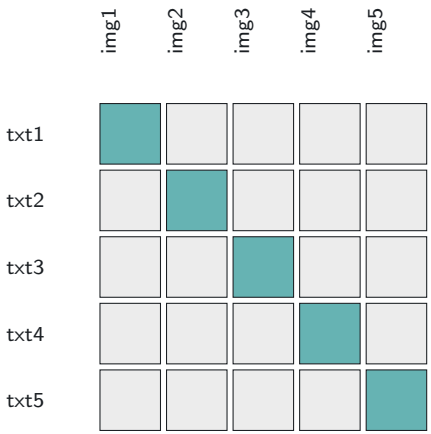
CLIP'S TRAINING OBJECTIVE, CONCRETELY / 2

- The N correct pairs are the matrix diagonal; every off-diagonal cell is a negative example that came *free* within the batch – no separate negative-sampling step needed, and larger batches give more negatives per pair for a sharper training signal (CLIP used batches of 32,768).
- **Zero-shot classification** in practice: turn each class name into a sentence (e.g. `"a photo of a {class}"`), embed all of them once, embed the query image, and pick the class whose text embedding has the highest similarity – no classification head, no gradient step, on data the model may never have seen labeled examples from.

AN INTUITION FOR CONTRASTIVE TRAINING

Picture a room with N people and N nametags, each nametag belonging to exactly one person – CLIP's training batch is exactly this "matching game", played on (image, caption) pairs instead.

AN INTUITION FOR CONTRASTIVE TRAINING / 2



AN INTUITION FOR CONTRASTIVE TRAINING / 3

- Every image is shown every caption in the batch, and gets a similarity score against each. Training pushes the correct pair (the diagonal, teal) toward the highest score in its row *and* its column, and every other combination (gray) toward a low score.
- The bigger the batch, the more "wrong nametags" each image is contrasted against per step, for free – which is exactly why CLIP trains with very large batches (tens of thousands of pairs): more negatives per positive means a sharper, more informative training signal.
- Nothing here is unique to images and text – the same trick trains a dual encoder for any two things you want a shared similarity space over (e.g. audio clips and their descriptions).

PATTERN 2: FUSION ARCHITECTURES

- Instead of two independent towers, run modality-specific encoders and then let a shared component attend across *both* – e.g. **UniT** (Hu & Singh, 2021): a BERT-style text encoder and a DETR-style vision encoder, whose outputs are concatenated into one sequence and fed to a shared decoder, together with a task-specific query.
- Trade-off versus dual encoders: slower (no pre-computable embeddings, the whole model runs per query), but able to model token-to-token interactions between modalities – necessary for tasks that need fine-grained grounding, e.g. visual question answering ("what color is the object left of the dog?" needs the model to actually align specific words with specific regions).

PATTERN 2: FUSION ARCHITECTURES / 2

- General pattern: **dual-encoder/contrastive when you need fast retrieval or zero-shot matching** (CLIP-style); **fusion when the task needs the two modalities to actually interact**.

Framing after the LMU Multimodal Deep Learning seminar (slds-lmu.github.io/seminar_multimodal_dl), licensed CC BY-NC-SA 4.0.

TELLING THE TWO PATTERNS APART, BY TASK

- **"Find me photos similar to this one" (reverse image search):** embed every photo in the database once, offline, with a dual encoder; at query time just embed the new photo and rank by similarity. Millions of comparisons per second, because nothing needs to be recomputed per query – this is precisely what a dual encoder is good at, and precisely why fusion would be far too slow here (it would have to rerun the whole model for every candidate).

TELLING THE TWO PATTERNS APART, BY TASK / 2

- **"What color is the object to the left of the dog?" (visual question answering):** answering this requires the model to locate "the dog", reason about "to the left of", and check color – all specific, fine-grained interactions between particular words and particular image regions. A dual encoder only ever produces two whole-image/whole-sentence summary vectors and compares them once at the end; by the time it does, the specific spatial relationship the question asks about has already been averaged away. Fusion, which lets word-tokens and patch-tokens attend to each other directly, is what actually has a chance at this.

TELLING THE TWO PATTERNS APART, BY TASK / 3

- The pattern generalizes past vision-language: it's the same distinction as document retrieval (dual-encoder embeddings, fast, coarse) versus a re-ranker that reads the query and each candidate document together (fusion, slow, precise) – systems in practice often use both in sequence, a dual encoder to shortlist candidates cheaply, then fusion to re-rank the shortlist accurately.

IMAGE CAPTIONING: ENCODER-DECODER, AGAIN

- Image captioning (image in, descriptive text out) is naturally an **encoder-decoder** task – the same pattern as T5 and Whisper, just with an image encoder instead of a text or audio one.
- History: CNN features → RNN decoder (pre-attention) → CNN features + an attention mechanism over spatial regions (*Show, Attend and Tell*, Xu et al., 2016 – already seen as a case study in session 7's Attention section) → today, a full Transformer: self-attention encoder over image-patch features, decoder with masked self-attention over words plus cross-attention over the encoder's output.

IMAGE CAPTIONING: ENCODER-DECODER, AGAIN / 2

One refinement worth naming: the **Meshed-Memory Transformer** (M^2, Cornia et al., 2020), a captioning-specific state-of-the-art at its time.

- **Memory-augmented attention** (encoder side): append extra, purely-learnable key/value vectors to the ones computed from the image – slots the model can use to store generic, dataset-level visual/semantic priors ("a plate usually has food on it") that aren't specific to any one image.

IMAGE CAPTIONING: ENCODER-DECODER, AGAIN / 3

- **Meshed cross-attention** (decoder side): instead of only cross-attending to the *final* encoder layer (as T5/Whisper do), attend to *every* encoder layer's output and combine them with learned, sigmoid-gated weights – low-level and high-level visual features both stay directly reachable from the decoder.
- Nothing here changes the encoder-decoder *pattern* from session 7 – both are extensions of where keys/values come from, not new attention mechanics.

TEXT-TO-IMAGE GENERATION: THE SHORT VERSION

- Text-to-image models (text in, image out) invert the captioning direction, and the connection to this session is the same either way: the text conditions image generation via **cross-attention** – the query-from-one-sequence / key-and-value-from-another-sequence recipe from session 7, wherever it appears in the pipeline.
- **Pre-Transformer**: text-conditioned GANs (StackGAN, AttnGAN) – a GAN generator conditioned on a text embedding, with attention (AttnGAN) letting individual image regions attend to individual words.

TEXT-TO-IMAGE GENERATION: THE SHORT VERSION / 2

- **DALL-E 1** (Ramesh et al., 2021): an autoregressive Transformer (GPT-style, session 9's family) generating discrete image tokens (from a VQ-VAE codebook) one at a time, conditioned on preceding text tokens – text-to-image as plain next-token prediction, at the cost of being slow to sample from.

TEXT-TO-IMAGE GENERATION: THE DIFFUSION ERA

- **GLIDE** and **DALL-E 2 (unCLIP)** switch to **diffusion models** (session 6): at every denoising step, the partially-generated image cross-attends to the text's token embeddings (GLIDE), or the diffusion model is conditioned on a CLIP image embedding predicted from the text embedding by a separate "prior" network (DALL-E 2's two-stage design) – CLIP's dual encoders (this session, Pattern 1) repurposed as a conditioning signal rather than for classification.

TEXT-TO-IMAGE GENERATION: THE DIFFUSION ERA / 2

- **Imagen** (Saharia et al., 2022) conditions diffusion directly on a large *frozen* pretrained **T5** text encoder – session 8's flagship encoder-decoder model, reused here purely for its encoder half – and finds a bigger, better-trained language model matters more for image quality than a bigger diffusion model.
- **Parti** (Yu et al., 2022) goes the other direction: autoregressive like DALL-E 1, but with a much larger ViT-VQGAN image tokenizer and model scale – the diffusion-vs-autoregressive split for image generation mirrors this whole course's encoder/decoder-family distinctions, not a settled question.

TEXT-TO-IMAGE GENERATION: THE DIFFUSION ERA / 3

- Open-source note: **Stable Diffusion** popularized running the diffusion process in a compressed *latent* space (a pretrained autoencoder's bottleneck) instead of raw pixels – the main reason it's cheap enough to run on a single consumer GPU.

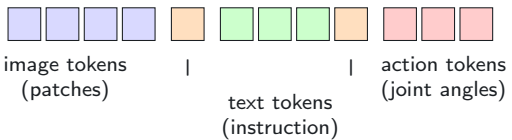
Framing after the LMU Multimodal Deep Learning seminar, "Text-to-Image Generation", CC BY-NC-SA 4.0.

Unified, Domain-Agnostic Architectures

ONE MODEL, ANY MODALITY: GATO

- **Gato** (Reed et al., 2022): a single **decoder-only** Transformer – the same family as GPT (session 9) – trained on text, images, robot joint-actions, and Atari game observations, all at once.
- The trick is entirely in the tokenization, not the architecture: every modality is serialized into a flat token sequence (images: ViT-style patches; continuous actions: discretized into tokens; a separator token | marks the boundary between an observation and the action taken in response), then it's just next-token prediction, exactly like any GPT-style model.
- If a Transformer decoder is just "predict the next token given the previous ones," and *anything* can be serialized into tokens, the architecture itself was never specific to language – language was simply the first domain where tokenization was obvious.

WHAT ONE GATO TRAINING EXAMPLE ACTUALLY LOOKS LIKE



- One single flat sequence: a camera image (patchified exactly like ViT), then a separator |, then a text instruction ("pick up the red block"), another separator, then a robot arm's next action (discretized into tokens, the same way a continuous pixel value or an image patch already gets discretized/embedded elsewhere in this course).

WHAT ONE GATO TRAINING EXAMPLE ACTUALLY LOOKS LIKE / 2

- The model has no idea (and no architectural mechanism to know) that some of these tokens "are" pixels and others "are" joint angles – to the Transformer, it is one sequence, and the loss is the same next-token prediction loss GPT uses on pure text.
- This is why Gato needs no vision-specific or robotics-specific submodule: all of the domain-specific work happens once, at tokenization, before the Transformer ever sees the data.

OFA: ONE SEQ2SEQ MODEL, ANY TASK

- **OFA** ("Once For All", Wang et al., 2022) takes the text-to-text idea from session 8's T5 and pushes it across modalities: *every* input and output – text, image patches, bounding-box coordinates – is sequentialized into one shared vocabulary and processed by a single encoder-decoder Transformer.
- Image patches are tokenized the same way DALL-E's discrete image tokens work (each patch maps to one of a fixed codebook of visual tokens), so images and text share one token space instead of needing separate embedding tables.

OFA: ONE SEQ2SEQ MODEL, ANY TASK

/ 2

- This unifies *understanding* tasks (captioning, VQA – output is text) and *generation* tasks (text-to-image – output is image tokens) as literally the same kind of seq2seq problem, distinguished only by a task-instruction prefix, exactly like T5's task prefixes.

THE LANDSCAPE, BRIEFLY

- **UniT** (fusion, seen above) and **OFA** (unified seq2seq, previous frame) both sequentialize modalities into a shared representation – they differ in whether the decoder is task-specific (UniT) or the task itself is described in the input (OFA).
- **LIMoE** (Mustafa et al., 2022): keeps modality-specific processing not via separate encoders, but via a **mixture-of-experts** layer – each token (regardless of modality) is routed to whichever few experts (out of many) suit it best, letting the model specialize internally without hand-splitting the architecture by modality.
- Common theme across all of these and Gato: the architectural primitives from session 7 (self-attention, positional encoding) stay fixed; what differs is only *how a given modality gets turned into tokens* in the first place.

THE LANDSCAPE, BRIEFLY / 2

Framing after the LMU Multimodal Deep Learning seminar, “Multi-Purpose Models”, CC BY-NC-SA 4.0.

Video: One More Dimension

THE PROBLEM: VIDEO ADDS A TIME AXIS

- A video is a sequence of frames – the most direct extension of ViT patchifies *space and time together*: a small spatial patch spanning a handful of consecutive frames (a "tubelet") becomes one token, instead of one flat image patch.
- This multiplies sequence length by the number of frames sampled, which makes the $\mathcal{O}(n^2)$ self-attention cost from session 7 far worse than for a single image – attending every space-time patch to every other one is usually not affordable.
- No new architectural primitive is needed to solve this – just session 7's efficient-attention toolbox (local/factorized patterns), applied to a harder instance of the same problem.

VIVIT: FOUR WAYS TO FACTOR SPACE AND TIME

Arnab et al., 2021 propose four variants, trading accuracy against cost:

- **Spatio-temporal attention**: attend jointly over every space-time patch. Best accuracy, cost $\mathcal{O}(n_h^2 n_w^2 n_t^2)$ – the expensive, unfactorized baseline.
- **Factorised encoder**: run an ordinary spatial ViT independently per frame, then a separate, small temporal Transformer over the resulting per-frame summary tokens. Cheaper ($\mathcal{O}(n_h^2 n_w^2 + n_t^2)$), because space and time are handled sequentially rather than jointly.

VIVIT: FOUR WAYS TO FACTOR SPACE AND TIME / 2

- **Factorised self-attention** and **factorised dot-product attention**: stay within one block, but split attention into a spatial pass and a temporal pass (either sequentially, or across different attention heads) rather than factorizing into two separate sub-networks.

VIVIT: FOUR WAYS TO FACTOR SPACE AND TIME / 3

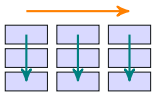
An intuition for factorizing: think about how *you* understand a movie scene.

Joint (spatio-temporal)



every patch attends to every other, in every frame, at once

Factorised



understand each frame first, *then* relate frames to each other

VIVIT: FOUR WAYS TO FACTOR SPACE AND TIME / 4

- **Joint attention** is like staring at all frames superimposed on top of each other simultaneously, trying to spot every relationship between every pixel in every frame at once – accurate in principle, but an enormous amount to process together.
- **Factorised attention** is what you actually do: first understand what's happening *within* each frame (spatial), then track how that understanding *changes across* frames (temporal) – two much smaller problems solved one after the other, instead of one huge problem solved all at once.

GETTING TOKENS FROM VIDEO

- **Uniform frame sampling**: take every k-th frame, patchify each independently, exactly like a batch of images – simple, but temporal information only enters through the attention pattern, never through tokenization itself.
- **Tubelet embedding**: each token's linear projection spans a small 3D space-time volume directly, so adjacent-frame information is fused *before* the first attention layer even runs – a strictly richer starting representation, at the cost of a slightly more involved patch-extraction step.

TIMESFORMER: DIVIDED SPACE-TIME ATTENTION

- Concurrent work ▸ Bertasius et al., 2021 tried five different attention schemes for video and found **divided space-time attention** the best trade-off: within each block, first attend over space (same frame, all positions), then over time (same position, all frames) – sequentially divided, not a single joint operation.
- Same underlying idea as ViViT's factorised variants, arrived at independently: joint spatio-temporal attention is rarely worth its cost once you can factor the two axes apart.

Framing after Hugging Face's Community Computer Vision Course, Unit 7 "Video Processing" (MIT licensed).

Closing Synthesis: Four Sessions, One Architecture

THE COMPLETE MAP

	Encoder-only	**Decoder-only**	**Encoder-decoder**
NLP	BERT, RoBERTa, ALBERT, DistilBERT, ELECTRA	GPT (session 9)	original Transformer, T5
Vision	Vision Transformer	autoregressive image generation	DETR (parallel, non-autoregressive decoding)
Audio	wav2vec 2.0-style encoders	–	Whisper
Cross-modal / unified	CLIP (dual-encoder)	Gato (any modality as tokens)	captioning, text-to-image cross-attention, UniT/OFA (fusion)

THE COMPLETE MAP / 2

- Four sessions, one small set of ideas: self-attention as content-based routing, positional encoding to recover order, and three attention patterns (bidirectional, causal, cross) that combine into three architecture families – everything else in this table is a choice of *what gets tokenized* and *which objective trains it*.
- Session 11 changes topic entirely: reinforcement learning.

REFERENCES


Dosovitskiy, A. et al. (2020). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. https://arxiv.org/abs/2010.11929


Carion, N. et al. (2020). *End-to-End Object Detection with Transformers (DETR)*. https://arxiv.org/abs/2005.12872


Zhu, X. et al. (2020). *Deformable DETR*. https://arxiv.org/abs/2010.04159


Meng, D. et al. (2021). *Conditional DETR for Fast Training Convergence*. https://arxiv.org/abs/2108.06152


Radford, A. et al. (2021). *Learning Transferable Visual Models From Natural Language Supervision (CLIP)*. https://arxiv.org/abs/2103.00020


Hu, R., & Singh, A. (2021). *UniT: Multimodal Multitask Learning with a Unified Transformer*. https://arxiv.org/abs/2102.10772

REFERENCES / 2


Xu, K. et al. (2016). *Show, Attend and Tell*.
https://arxiv.org/abs/1502.03044


Saharia, C. et al. (2022). *Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding (Imagen)*.
https://arxiv.org/abs/2205.11487


Reed, S. et al. (2022). *A Generalist Agent (Gato)*.
https://arxiv.org/abs/2205.06175


Wang, P. et al. (2022). *OFA: Unifying Architectures, Tasks, and Modalities Through a Simple Sequence-to-Sequence Learning Framework*.
https://arxiv.org/abs/2202.03052


Mustafa, B. et al. (2022). *Multimodal Contrastive Learning with LIMoE*.
https://arxiv.org/abs/2206.02770

REFERENCES / 3


Redmon, J. et al. (2015). *You Only Look Once (YOLO)*. https://arxiv.org/abs/1506.02640


Liu, Z. et al. (2021). *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows*. https://arxiv.org/abs/2103.14030


Touvron, H. et al. (2021). *Training Data-Efficient Image Transformers & Distillation Through Attention (DeiT)*. https://arxiv.org/abs/2012.12877


Cheng, B., Schwing, A., & Kirillov, A. (2021). *Per-Pixel Classification is Not All You Need for Semantic Segmentation (MaskFormer)*. https://arxiv.org/abs/2107.06278


Arnab, A. et al. (2021). *ViViT: A Video Vision Transformer*. https://arxiv.org/abs/2103.15691

REFERENCES / 4


Bertasius, G., Wang, H., & Torresani, L. (2021). *Is Space-Time Attention All You Need for Video Understanding? (TimeSformer)*.
https://arxiv.org/abs/2102.05095


Cornia, M. et al. (2020). *Meshed-Memory Transformer for Image Captioning*.
https://arxiv.org/abs/1912.08226


Ramesh, A. et al. (2021). *Zero-Shot Text-to-Image Generation (DALL-E)*.
https://arxiv.org/abs/2102.12092


Nichol, A. et al. (2021). *GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models*.
https://arxiv.org/abs/2112.10741


Ramesh, A. et al. (2022). *Hierarchical Text-Conditional Image Generation with CLIP Latents (DALL-E 2 / unCLIP)*.
https://arxiv.org/abs/2204.06125

REFERENCES / 5



Yu, J. et al. (2022). *Scaling Autoregressive Models for Content-Rich Text-to-Image Generation (Parti)*. https://arxiv.org/abs/2206.10789



Rombach, R. et al. (2022). *High-Resolution Image Synthesis with Latent Diffusion Models (Stable Diffusion)*. https://arxiv.org/abs/2112.10752

License & Attribution

LICENSE & ATTRIBUTION

- Unlike sessions 7-8, no content in this session is adapted from i2dl or DL4NLP (neither course covers non-NLP Transformer applications). All frames are freshly written for this course, since none of the source material below is in Beamer slide form.
- The "Vision Transformers, In Depth", "Efficient Vision Transformers" (Swin, DeiT), "DETR", and "Segmentation Transformers" sections follow the explanations (and, for the DETR code frame, the code) of the **Hugging Face Community Computer Vision Course** (`https://github.com/huggingface/computer-vision-course`) — licensed **MIT**, verified directly from the repository's license file. DeiT's specific mechanism (the distillation token) is stated from general knowledge of the original paper, not from the course's

LICENSE & ATTRIBUTION / 2

own (more general) knowledge-distillation chapter, which does not describe DeiT specifically.

- The "Video" section (ViViT's four variants, tubelet embedding, TimeSformer) also follows the same Hugging Face course, Unit 7 "Video Processing" — same MIT license.
- The "Combining Modalities" and "Unified, Domain-Agnostic Architectures" sections (incl. OFA) follow the **LMU Multimodal Deep Learning seminar** (`https://slds-lmu.github.io/seminar_multimodal_dl/`) — licensed **CC BY-NC-SA 4.0** (confirmed in the book's own front matter), the same license class already carried by session 7's Fleuret-derived material: non-commercial academic teaching is covered, but reuse must be attributed and shared alike.

LICENSE & ATTRIBUTION / 3

- All diagrams (patch-embedding illustration, DETR architecture, dual-encoder diagram) are original TikZ, not reproductions of any source's figures — both courses' own images were left out entirely rather than downloaded, since their individual provenance (some may originate from the original papers rather than the course authors) wasn't independently verified.
- Stanford's CS25 ("Transformers United") was checked for this session's scope (topic list only) and confirmed to carry *no* license or reuse statement of any kind – no content, code, or figures from it are used anywhere in this deck.
- The closing synthesis table is original content, not adapted from any single source.

LICENSE & ATTRIBUTION / 4

- All worked examples and intuitive analogies added in this session's third pass (the patch-count arithmetic, the shifted-photo inductive-bias example, Swin's paragraph-shifting analogy, DETR's numeric matching example, CLIP's "matching game" framing and similarity-matrix diagram, the dual-encoder-vs-fusion task comparison, Gato's token-sequence diagram, and ViViT's movie-watching analogy) are freshly authored for this course, not sourced from HF or LMU – neither course phrases these subtopics this way. Where an added frame restates a fact already cited elsewhere in this deck (e.g. the $224 \times 224/16 \times 16$ patch arithmetic follows directly from Dosovitskiy et al.'s own patch-size choice), the original citation from the earlier frame still applies; it is not repeated as a separate claim.